



Universidad
Tecmilenio®

NOMBRE: JOAQUIN RAUL NAJERA SALDAÑA

MATRICULA: 7132489

MAESTRO: Edgar Ivan Patricio Aizpuru

Actividad 3 Estructuras de datos Escape de laboratorio

Introducción

En esta actividad se desarrolló un programa en Java para resolver tres retos relacionados con técnicas importantes de programación y estructuras de datos. El propósito principal fue aplicar recursividad, divide y vencerás y backtracking dentro de una misma solución, pero en problemas diferentes.

El primer reto consistió en reducir varios números hasta obtener un solo dígito utilizando llamadas recursivas. El segundo reto se enfocó en encontrar el valor máximo de un arreglo mediante la estrategia divide y vencerás, evitando métodos de ordenamiento o recorridos tradicionales para buscar directamente el mayor elemento. Finalmente, el tercer reto consistió en resolver un laberinto generado por un sistema externo, utilizando backtracking para explorar diferentes caminos y determinar cuál era la ruta más corta entre el punto de inicio y la meta.

Para el tercer desafío se utilizó un archivo llamado maze.txt, descargado desde el Sistema de Escape. De esta manera, el programa no depende de un laberinto específico escrito dentro del código, sino que puede leer diferentes escenarios y resolverlos de forma automática.

Además de obtener resultados correctos, se buscó que el programa pudiera controlar situaciones como obstáculos, posiciones ya visitadas, ciclos, caminos sin salida, diferentes rutas posibles y casos en los que no existe una solución.

Desarrollo

Desafío 1

El primer desafío consiste en aplicar recursividad para reducir cada uno de los siguientes números hasta obtener un solo dígito:

84729

56318

92746

Para realizar este procedimiento se creó un método que suma los dígitos del número mediante llamadas recursivas.

El proceso continúa hasta que el resultado tiene solamente un dígito.

Primer bloque

Para el número:

84729

se realiza la suma:

$$8 + 4 + 7 + 2 + 9 = 30$$

El resultado todavía tiene dos dígitos, por lo que se vuelve a aplicar el proceso:

$$3 + 0 = 3$$

Resultado:

$$84729 \rightarrow 3$$

Segundo bloque

Para el número:

56318

se obtiene:

$$5 + 6 + 3 + 1 + 8 = 23$$

Después:

$$2 + 3 = 5$$

Resultado:

$$56318 \rightarrow 5$$

Tercer bloque

Para:

92746

se realiza:

$$9 + 2 + 7 + 4 + 6 = 28$$

Después:

$$2 + 8 = 10$$

Finalmente:

$$1 + 0 = 1$$

Resultado:

92746 → 1

Los tres resultados se conservan en el mismo orden:

3

5

1

Por lo tanto:

CLAVE 1 = 351

Caso base del desafío 1

El caso base se presenta cuando el número recibido tiene solamente un dígito.

En el programa se utiliza una condición equivalente a:

```
if (numero < 10) {  
    return numero;  
}
```

Cuando se cumple esta condición, el método deja de realizar llamadas recursivas.

Caso recursivo del desafío 1

El caso recursivo se utiliza cuando el número contiene más de un dígito.

Para sumar los dígitos se obtiene el último mediante:

$\text{numero \% 10}$

y el resto del número se obtiene utilizando:

numero / 10

El método vuelve a llamarse utilizando la parte restante del número.

Conceptualmente:

$\text{sumarDigitos}(1234)$

$4 + \text{sumarDigitos}(123)$

$4 + 3 + \text{sumarDigitos}(12)$

4 + 3 + 2 + sumarDigitos(1)

De esta forma la suma se realiza sin utilizar ciclos.

Desafío 2 El radar de emergencia

En el segundo desafío se proporcionó el siguiente conjunto de lecturas:

[418, 732, 156, 894, 327, 641, 285, 519,
763, 204, 947, 386, 675, 128, 856, 493]

El objetivo fue encontrar la lectura más alta utilizando la estrategia divide y vencerás.

No se utilizaron métodos como:

`Arrays.sort()`

`Collections.max()`

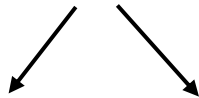
ni se ordenó previamente el arreglo.

División del problema

Primero, el arreglo se divide en dos partes.

Conceptualmente:

Arreglo completo



Mitad izquierda Mitad derecha

Cada mitad vuelve a dividirse de forma recursiva hasta que cada subproblema contiene solamente un elemento.

Resolución

Cuando un segmento contiene un solo elemento, ese elemento se considera el máximo de ese pequeño subproblema.

Por ejemplo:

[418]

El máximo es:

418

En este punto se alcanza el caso base de la recursividad.

Combinación

Después se comparan los máximos obtenidos de ambos lados.

Por ejemplo:

máximo izquierda = 894

máximo derecha = 947

Entonces:

`Math.max(894, 947)`

produce:

947

Al finalizar todas las comparaciones, el valor máximo del arreglo es:

947

Por lo tanto:

CLAVE 2 = 947

Obtención de la contraseña

Las dos claves obtenidas fueron:

CLAVE 1 = 351

CLAVE 2 = 947

El formato solicitado fue:

CLAVE1-CLAVE2

Por lo tanto, la contraseña final fue:

351-947

Esta contraseña fue utilizada para ingresar al Sistema de Escape.

Desafío 3 La ruta de escape

Después de ingresar correctamente al Sistema de Escape, el sistema genera un laberinto aleatorio.

En el escenario utilizado se manejan los siguientes caracteres:

S = Inicio

T = Meta

= Pared

. = Camino

El programa también permite utilizar E como meta, para conservar compatibilidad con las instrucciones generales de la actividad.

El objetivo del tercer desafío no consiste únicamente en encontrar un camino válido.

El programa debe encontrar:

LA RUTA MÁS CORTA

Lectura del archivo maze.txt

Para trabajar con el laberinto generado por el sistema se utilizó el archivo:

maze.txt

Este archivo se descarga directamente desde la página del Sistema de Escape.

Posteriormente se coloca en la misma carpeta del proyecto:

Actividad3/

|

|— Main.java

|— maze.txt

El programa lee automáticamente el contenido del archivo utilizando Java.

Esto permite que el código funcione con diferentes laberintos sin tener que cambiar manualmente las posiciones o la ruta.

Identificación del inicio y de la meta

Una vez cargado el laberinto, el programa busca automáticamente la posición:

S

que representa el inicio.

Posteriormente busca:

T

como meta.

Si el escenario utiliza:

E

el programa también puede reconocerlo.

Por esta razón no es necesario conocer previamente las coordenadas del inicio o de la salida.

Funcionamiento del backtracking

El backtracking es una técnica que permite probar diferentes alternativas.

El algoritmo empieza desde la posición inicial y analiza los movimientos disponibles.

Desde cada posición intenta desplazarse en cuatro direcciones:

Arriba

Abajo

Izquierda

Derecha

Si una dirección es válida, el programa avanza y continúa la búsqueda de forma recursiva.

Cuando una ruta no puede continuar, el algoritmo regresa a una posición anterior y prueba otra alternativa.

Este proceso continúa hasta explorar las posibilidades necesarias.

Control de posiciones inválidas

Antes de avanzar, el programa verifica que la nueva posición se encuentre dentro del laberinto.

Esto evita intentar acceder a posiciones fuera de los límites de la matriz.

También verifica si la posición contiene una pared:

#

Si encuentra una pared, ese movimiento se descarta.

Control de ciclos

Durante la búsqueda puede existir la posibilidad de regresar varias veces a la misma posición.

Por ejemplo:

$A \rightarrow B$

↑ ↓

$D \leftarrow C$

Si el programa no controla este comportamiento podría entrar en un ciclo.

Para evitarlo se utiliza una matriz booleana llamada:

visitado

Cuando una posición forma parte de la ruta actual, se marca como visitada.

Si el algoritmo intenta volver a la misma posición antes de terminar ese camino, no puede avanzar.

Desmarcar posiciones durante el backtracking

Una característica importante de la solución es que las posiciones no permanecen marcadas para siempre.

Cuando el algoritmo regresa de un camino, se realiza:

visitado = false

para esa posición.

Esto permite que otra ruta diferente pueda utilizarla posteriormente.

Gracias a esta estrategia se evitan ciclos dentro de una ruta, pero al mismo tiempo se pueden explorar diferentes soluciones.

Ruta actual

Mientras el algoritmo explora el laberinto se guarda una lista denominada:

rutaActual

En esta lista se van almacenando las posiciones recorridas.

Por ejemplo:

0,0

1,0

2,0

2,1

3,1

Cuando el algoritmo debe retroceder, elimina la última posición de esta lista.

Retroceso

Cuando un camino no produce una solución, el programa vuelve a la posición anterior.

Este proceso es precisamente el backtracking.

Conceptualmente:

Avanzar



Probar camino



No funciona



Regresar



Probar otra alternativa

Cada vez que el programa retrocede también se incrementa la métrica:

retrocesos

Determinación de la ruta más corta

La actividad indica que encontrar la primera ruta no es suficiente.

Por esta razón el algoritmo no se detiene cuando llega por primera vez a la meta.

Cuando se encuentra una ruta válida, se almacena como la mejor ruta disponible hasta ese momento.

Posteriormente, el algoritmo continúa explorando otros caminos.

Si encuentra otra ruta con una cantidad menor de posiciones, reemplaza la ruta anterior.

De esta manera se conserva siempre:

mejorRuta

Cálculo de movimientos

La cantidad de movimientos se calcula tomando en cuenta el número de posiciones recorridas.

Por ejemplo:

0,0

0,1

0,2

0,3

existen cuatro posiciones, pero solamente tres movimientos.

Por esta razón:

$\text{movimientos} = \text{cantidad de posiciones} - 1$

Poda de búsqueda

Para mejorar el rendimiento del algoritmo también se implementó una poda.

La idea consiste en dejar de explorar una alternativa cuando ya se sabe que no puede mejorar la mejor solución encontrada.

Por ejemplo, si ya existe una ruta de:

20 movimientos

y una nueva ruta ya tiene una longitud igual o superior sin haber llegado a la salida, esa alternativa no puede convertirse en una ruta mejor.

Por lo tanto, la exploración de esa rama se detiene.

Esto permite reducir búsquedas innecesarias.

Posiciones visitadas

Durante la exploración se registran las posiciones procesadas por el programa.

La salida utiliza el formato solicitado:

fila,columna

Por ejemplo:

0,0

1,0

2,0

2,1

2,2

Estas posiciones permiten comprobar que el programa realmente realiza una exploración del laberinto.

Ruta final más corta

Al terminar la búsqueda, el programa muestra la ruta final utilizando el siguiente formato:

RUTA:r0,c0;r1,c1;r2,c2;...

Ejemplo:

RUTA:0,0;1,0;2,0;2,1;3,1;4,1

También se muestra la cantidad total de movimientos.

Métricas

El programa muestra dos métricas principales.

Retrocesos

Representa la cantidad de veces que el algoritmo tuvo que regresar durante el proceso de backtracking.

Estados

Representa la cantidad de estados o posiciones procesadas durante la búsqueda.

El formato utilizado es:

METRICAS:retrocesos=X;estados=Y

Los valores cambian dependiendo del laberinto generado por el sistema.

Pruebas realizadas

Para comprobar que la solución no funciona únicamente con un escenario específico, se consideraron distintos tipos de pruebas.

Caso 1 Ruta directa

En este escenario existe una ruta sencilla entre el inicio y la meta.

Ejemplo:

S...T

El algoritmo debe avanzar directamente y encontrar la ruta con menor cantidad de movimientos.

Caso 2 Callejón sin salida

En esta prueba existe un camino que parece válido inicialmente, pero termina bloqueado.

El programa debe detectar que no puede continuar, realizar un retroceso y probar otra ruta.

Esta prueba permite comprobar el funcionamiento correcto del backtracking.

Caso 3 Varias rutas

El laberinto contiene más de un camino válido para llegar a la meta.

El algoritmo debe explorar diferentes alternativas y comparar sus longitudes.

Al finalizar conserva solamente la ruta más corta.

Caso 4 La primera ruta no es la mejor

En esta prueba el primer camino encontrado sí llega a la salida, pero existe otra ruta más corta.

El algoritmo debe continuar explorando después de encontrar la primera solución.

Si encuentra una mejor ruta, reemplaza la anterior.

Caso 5 Sin solución

También se considera el caso en el que no existe ninguna ruta desde el inicio hasta la meta.

Por ejemplo:

S#T

###

...

En este escenario el programa debe mostrar:

SIN SOLUCION

Esto demuestra que el algoritmo puede identificar cuando no existe un camino válido.

¿Cuál es el caso base del desafío 1?

El caso base ocurre cuando el número tiene solamente un dígito.

En ese momento el método devuelve el número directamente y la recursividad termina.

¿Cuál es el caso recursivo?

Cuando el número tiene más de un dígito, el método separa el último dígito y vuelve a llamarse utilizando el resto del número.

Al finalizar se suman los resultados parciales.

¿Cómo se aplica divide y vencerás en el desafío 2?

El arreglo se divide en dos partes.

Cada parte se resuelve de manera recursiva.

Finalmente se comparan los máximos obtenidos de ambos lados.

¿Cómo se divide el problema?

Se obtiene un punto medio dentro del arreglo.

Una llamada recursiva procesa la primera mitad y otra procesa la segunda mitad.

¿Cómo se combinan los resultados?

Después de obtener el valor máximo de ambas mitades, se utiliza una comparación para elegir el mayor.

De esta manera los resultados parciales se van combinando hasta obtener el máximo del arreglo completo.

¿Cómo funciona el backtracking del desafío 3?

El programa prueba una posición y continúa explorando desde ella.

Si el camino no funciona, regresa a la posición anterior y prueba otra dirección.

Este proceso se repite hasta encontrar y comparar las rutas posibles.

¿Cómo se evitan ciclos?

El programa utiliza una matriz para identificar las posiciones que forman parte de la ruta actual.

No se permite avanzar hacia una posición que ya está marcada.

¿Cómo se determina cuál ruta es la más corta?

Cada vez que el algoritmo llega a la meta se compara la longitud de la nueva ruta con la mejor solución almacenada.

Si la nueva ruta es más corta, reemplaza a la anterior.

¿Qué optimización se aplicó y por qué?

Se implementó una poda de búsqueda.

Si una ruta parcial ya tiene una longitud igual o superior a la mejor solución encontrada, se deja de explorar esa alternativa.

Esto ayuda a reducir trabajo innecesario.

Reflexión final

1. ¿Qué diferencia existe entre una solución recursiva y una solución iterativa?

Una solución recursiva utiliza métodos que se llaman a sí mismos hasta llegar a una condición de salida.

Una solución iterativa utiliza estructuras repetitivas como for, while o do-while.

La recursividad puede ser especialmente útil cuando el problema puede dividirse en versiones más pequeñas del mismo problema.

2. ¿Por qué divide y vencerás puede facilitar la resolución de ciertos problemas?

Porque permite separar un problema grande en partes pequeñas que son más fáciles de resolver.

Después, las soluciones de esas partes se combinan para obtener la respuesta final.

3. ¿Por qué encontrar una solución con backtracking no significa necesariamente que sea la mejor solución?

Porque la primera ruta encontrada puede ser válida, pero puede existir otra ruta con menos movimientos.

Por esa razón es necesario seguir explorando y comparar las soluciones.

4. ¿Qué problema podría presentarse si el algoritmo no controla las posiciones ya visitadas?

Podría entrar en ciclos y regresar repetidamente a las mismas posiciones.

Esto puede provocar una cantidad excesiva de llamadas recursivas o impedir que el programa termine correctamente.

5. ¿Qué aprendiste al tener que encontrar la ruta más corta en lugar de simplemente encontrar una ruta?

Aprendí que en algunos problemas no basta con encontrar una respuesta válida.

También es necesario comparar diferentes soluciones para determinar cuál es la más eficiente.

Además, fue importante comprender el uso del backtracking y la poda para reducir búsquedas innecesarias.

Evidencias de funcionamiento

Resultado del desafío 1

```
DESAFIO 1: RECURSIVIDAD
84729 -> 3
56318 -> 5
92746 -> 1
CLAVE 1: 351
```

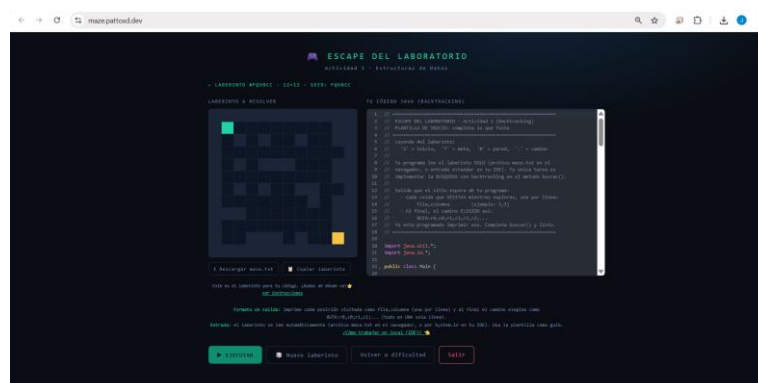
Resultado obtenido mediante la reducción recursiva de los tres bloques numéricos.

Resultado del desafío 2

```
DESAFIO 2: DIVIDE Y VENCERÁS
CLAVE 2: 947
CONTRASEÑA: 351-947
```

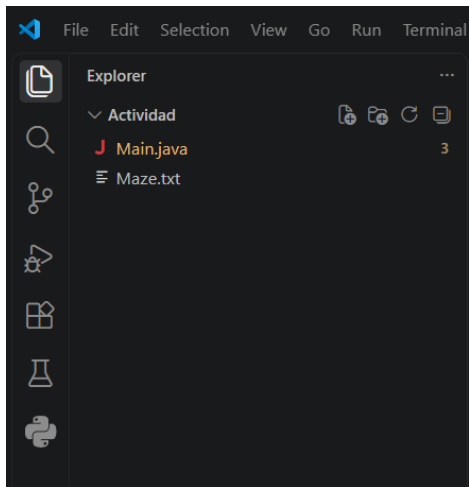
Resultado del algoritmo divide y vencerás y contraseña generada para ingresar al Sistema de Escape.

Sistema de Escape



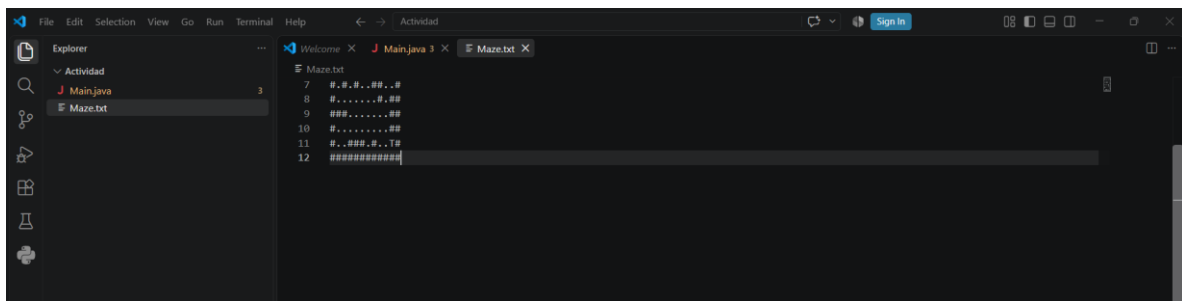
Escenario generado automáticamente después de ingresar correctamente la contraseña obtenida.

Archivo maze.txt



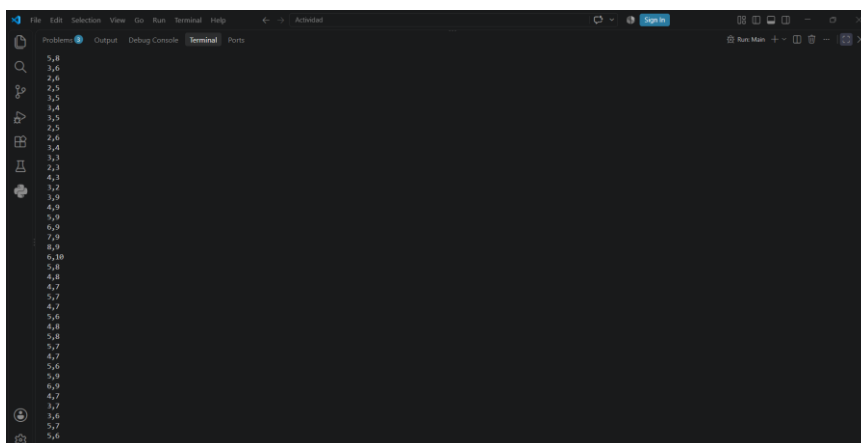
Archivo utilizado como entrada para el algoritmo de backtracking.

Laberinto cargado en Java



Comprobación de que Java obtiene automáticamente el escenario desde el archivo descargado.

Posiciones visitadas



Estados explorados durante la ejecución del algoritmo de backtracking.

Ruta final más corta

```
RUTA FINAL MAS CORTA:  
RUTA:1,1;2,1;3,1;4,1;5,1;6,1;7,1;7,2;7,3;8,3;9,3;9,4;9,5;9,6;9,7;9,8;10,8;10,9;10,10  
Movimientos: 18
```

Ruta más corta encontrada después de explorar las diferentes posibilidades.

Métricas

```
METRICAS:retrocesos=28971;estados=28982  
PS C:\Users\najer_6dww04m\Downloads\Actividad>
```

Cantidad de retrocesos y estados procesados durante la búsqueda.

Conclusión

El desarrollo de esta actividad permitió aplicar tres formas diferentes de resolver problemas utilizando programación. Aunque los tres desafíos forman parte de un mismo ejercicio, cada uno requiere una manera distinta de pensar la solución.

En el primer reto, la recursividad permitió trabajar con un número reduciéndolo poco a poco hasta llegar a una condición sencilla. En el segundo, divide y vencerás mostró cómo un problema grande puede separarse en partes más pequeñas para después combinar los resultados. En el último desafío, el backtracking permitió explorar un laberinto tomando decisiones, regresando cuando un camino no servía y probando nuevas alternativas.

La parte más importante del tercer desafío fue que el programa no se limitó a encontrar cualquier salida. Fue necesario comparar diferentes rutas y conservar la más corta. También fue importante controlar las posiciones ya visitadas para evitar ciclos y aplicar una poda para no seguir explorando caminos que ya no podían mejorar la solución obtenida.

Además, la lectura del archivo maze.txt permitió hacer el programa más general, ya que el laberinto no se encuentra escrito directamente dentro del código. Esto permite utilizar el mismo programa con diferentes escenarios generados por el sistema.

En general, la actividad ayudó a comprender mejor cómo la recursividad puede utilizarse en distintos tipos de problemas y cómo técnicas como divide y vencerás y backtracking permiten organizar la búsqueda de soluciones de una manera más estructurada y eficiente.

